



SISTEMAS DISTRIBUIDOS

Task1:Communication and middleware

2025-26

SALMA JADIANI

ALEJANDRO FERNÁNDEZ

ÍNDICE

1. Introducción

2. Diseño del Sistema

2.1. Modelos de tickets

2.2. Arquitecturas

2.3. Consistencia y Redis

3. Despliegue (2 máquinas).

4. Resultados y gráficas.

5. Conclusiones

1.Introducción

- **Objetivos:**

La gestión de venta de entradas para eventos de alta demanda (conciertos, eventos deportivos, etc.) representa uno de los desafíos más críticos en el diseño de sistemas distribuidos. El problema fundamental radica en la **conurrencia masiva**: miles de usuarios intentando adquirir un inventario limitado de asientos en el mismo segundo.

En esta práctica, se han implementado y analizado dos arquitecturas de comunicación para resolver este escenario:

1. **Comunicación Directa:** Basada en Pyro4 como middleware de RPC síncrono, con workers registrados en un Name Server y un balanceador de carga round-robin en el lado del server para distribuir las peticiones.
2. **Comunicación Indirecta:** Basada en un modelo asíncrono utilizando **RabbitMQ** como middleware de mensajería, donde el procesamiento se desacopla de la recepción mediante el uso de *Workers*.

El sistema utiliza **Redis** como motor de persistencia en disco (redis_conf.txt), aprovechando sus capacidades atómicas para garantizar que el estado de los tickets sea consistente en todo momento.

El desarrollo de este proyecto se ha guiado por los siguientes objetivos principales:

- **Garantizar la Consistencia de Datos:** Asegurar que, bajo ninguna circunstancia, un mismo asiento numerado sea vendido a dos clientes distintos (evitar el *overselling*).
- **Implementar Robustez y Resiliencia:** Diseñar el sistema de comunicación indirecta de modo que sea capaz de sobrevivir a fallos críticos. Esto incluye la persistencia de mensajes en la cola y el manejo de confirmaciones manuales para evitar la pérdida de transacciones si un procesador (*Worker*) se desconecta.
- **Analizar la Escalabilidad:** Evaluar cómo el rendimiento del sistema (medido en operaciones por segundo o *throughput*) mejora de forma dinámica al añadir más nodos de procesamiento.
- **Estudiar el Impacto de la Contención (Hotspots):** Observar y documentar el comportamiento del sistema cuando una gran mayoría de las peticiones se concentran en un subconjunto muy pequeño de recursos (asientos específicos), provocando cuellos de

botella en la base de datos.

- **Comparativa Crítica:** Determinar las ventajas y desventajas de cada modelo de comunicación en función de la carga de trabajo y los requisitos de disponibilidad.

2. Diseño del Sistema

2.1 Modelos de tickets

- **No numerados.**

Este modelo representa eventos donde las entradas no tienen un asiento asignado (ej. pista en un concierto).

- **Lógica de Implementación:** Se utiliza un contador atómico en Redis mediante el comando **INCR**.

```
tickets_vendidos = self.redis.incr("contador_tickets")
```

- **Gestión de Concurrencia:** El sistema comprueba si el valor del contador es inferior al aforo total antes de confirmar la venta.

```
tickets_vendidos <= 20000:
```

- **Rendimiento:** Al no haber colisiones por un recurso específico, la contención es mínima, permitiendo que múltiples Workers incrementen el contador simultáneamente sin bloqueos.
 - Numerados.

Este modelo es propenso a generar **Hotspots**, ya que cada petición solicita un identificador de asiento

- **Lógica de Implementación:** Se utiliza una estructura de clave-valor donde la clave es el identificador del asiento. La reserva se realiza mediante la operación atómica SET NX (Set if Not Exists).

```
reservado = self.redis.set(clave_asiento, cliente_id, nx=True)
```

- **Garantía de Exclusividad:** El uso de NX asegura que solo la primera petición que llegue a Redis consiga escribir su request_id en esa clave. Cualquier intento posterior para el mismo asiento será rechazado por la base de datos, devolviendo un error de "asiento ya vendido".
- **Impacto en el Diseño:** En los benchmarks de alta carga, muchas peticiones compiten por el mismo asiento (Hotspot), lo que obliga a

los Workers a procesar y descartar rápidamente las peticiones duplicadas para mantener la consistencia.

2.2 Arquitecturas

- **Directa (PYRO4)**

En este modelo, un objeto en una máquina (servidor) expone sus métodos para que puedan ser ejecutados por otra máquina (cliente) de forma transparente, como si fueran llamadas locales. Basado en RPC (*Remote Procedure Calls*). Este modelo permite que un cliente invoque métodos de objetos remotos de forma transparente, como si fueran llamadas locales.

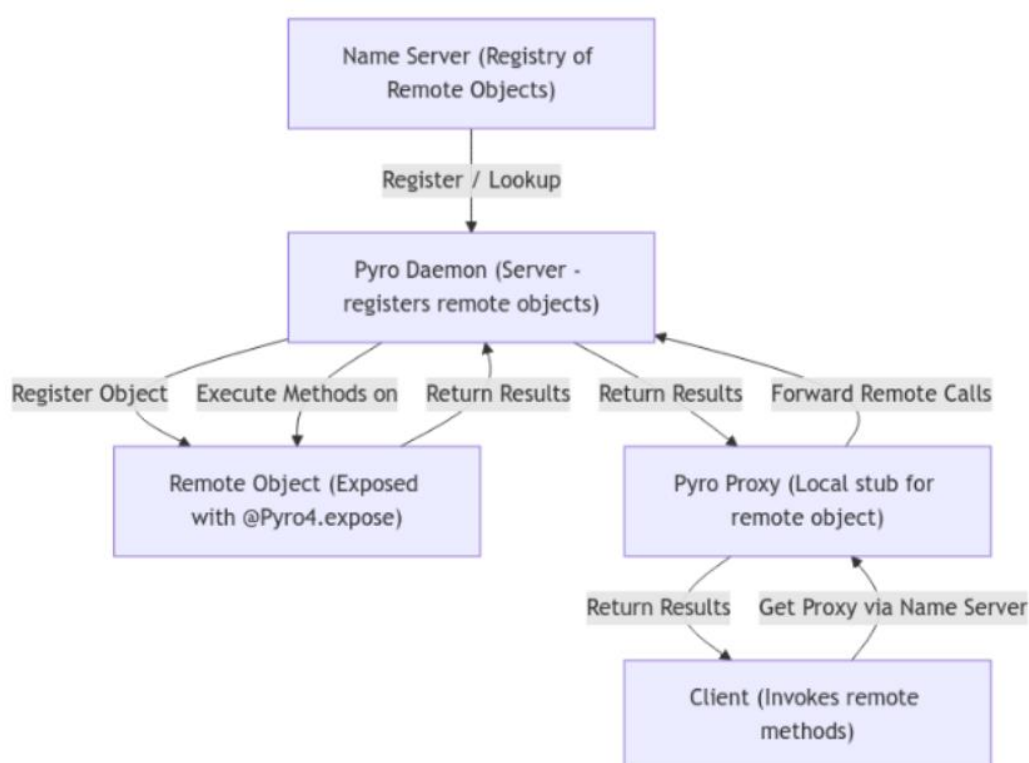


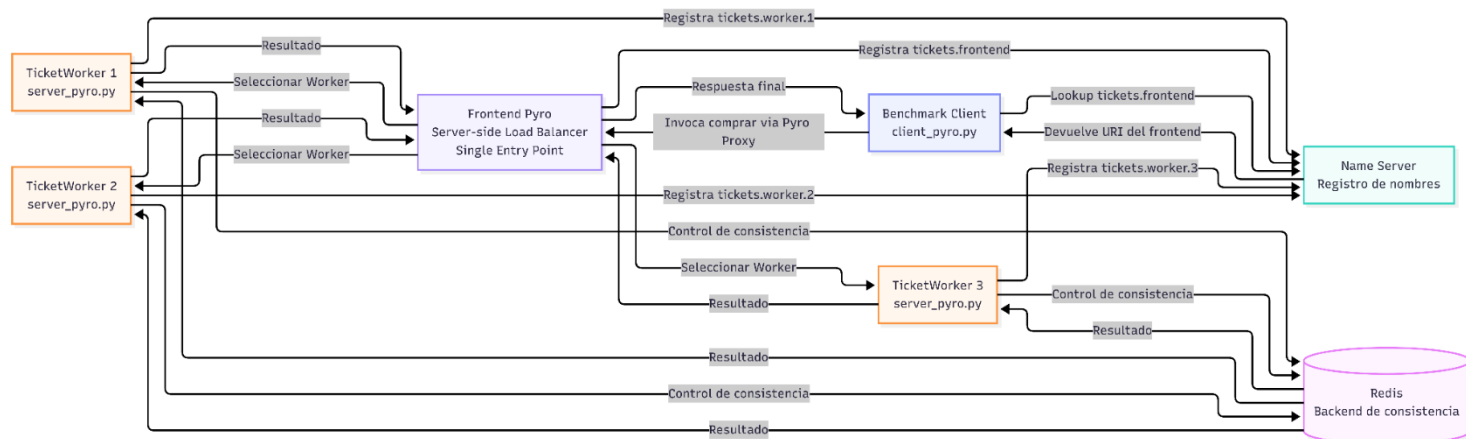
Fig. 1 - Arquitectura de Comunicación Directa (Pyro4)

Los componentes clave definidos en la teoría y aplicados en el proyecto son:

- **Name Server:** servicio encargado de registrar y localizar los objetos remotos.
- **Pyro Proxy:** proxy local que utiliza el cliente para comunicarse con los objetos remotos.
- **Pyro Daemon:** proceso servidor que recibe y gestiona las

peticiones RPC.

- **Remote Objects:** objetos remotos expuestos mediante el decorador `@Pyro4.expose`.



La arquitectura implementada se organiza en varios niveles.

Name Server

El sistema utiliza el Name Server de Pyro4 para desacoplar la ubicación física de los servicios. Tanto el frontend como los distintos workers se registran dinámicamente utilizando identificadores únicos:

- tickets.frontend
- tickets.worker.1
- tickets.worker.2
- tickets.worker.3

Gracias a este mecanismo, es posible añadir o eliminar workers sin necesidad de modificar el cliente.

Frontend Pyro (Single Entry Point)

La arquitectura implementa un único punto de entrada (tickets.frontend) que actúa como balanceador de carga del lado del servidor (*Server-side Load Balancer*).

El cliente únicamente consulta el objeto tickets.frontend en el Name Server y envía todas las peticiones a dicho frontend. Posteriormente, el frontend obtiene la lista de workers disponibles, selecciona uno mediante

un algoritmo *Round Robin* y reenvía la petición RPC al worker correspondiente.

Este enfoque permite desacoplar completamente el cliente de los workers internos del sistema y facilita el escalado dinámico de la arquitectura.

Workers

El archivo `server_pyro.py` implementa la clase `TicketWorker`, marcada con `@Pyro4.expose`.

Cada worker recibe las peticiones RPC enviadas por el frontend, ejecuta la lógica de compra y se comunica con Redis para garantizar la consistencia del sistema y evitar situaciones de sobreventa.

Backend de Consistencia (Redis)

Redis se utiliza como backend de consistencia mediante operaciones atómicas:

- INCR para tickets no numerados.
- SET NX para tickets numerados.

Estas operaciones permiten garantizar:

- ausencia de doble venta,
- consistencia bajo concurrencia,
- idempotencia de peticiones.

Cliente Benchmark

El archivo `client_pyro.py` se encarga de cargar los benchmarks y generar las peticiones concurrentes utilizando múltiples hilos.

El cliente únicamente interactúa con el frontend (`tickets.frontend`), manteniendo el desacoplamiento respecto a los workers internos del sistema.

- Indirecta (RabbitMQ)

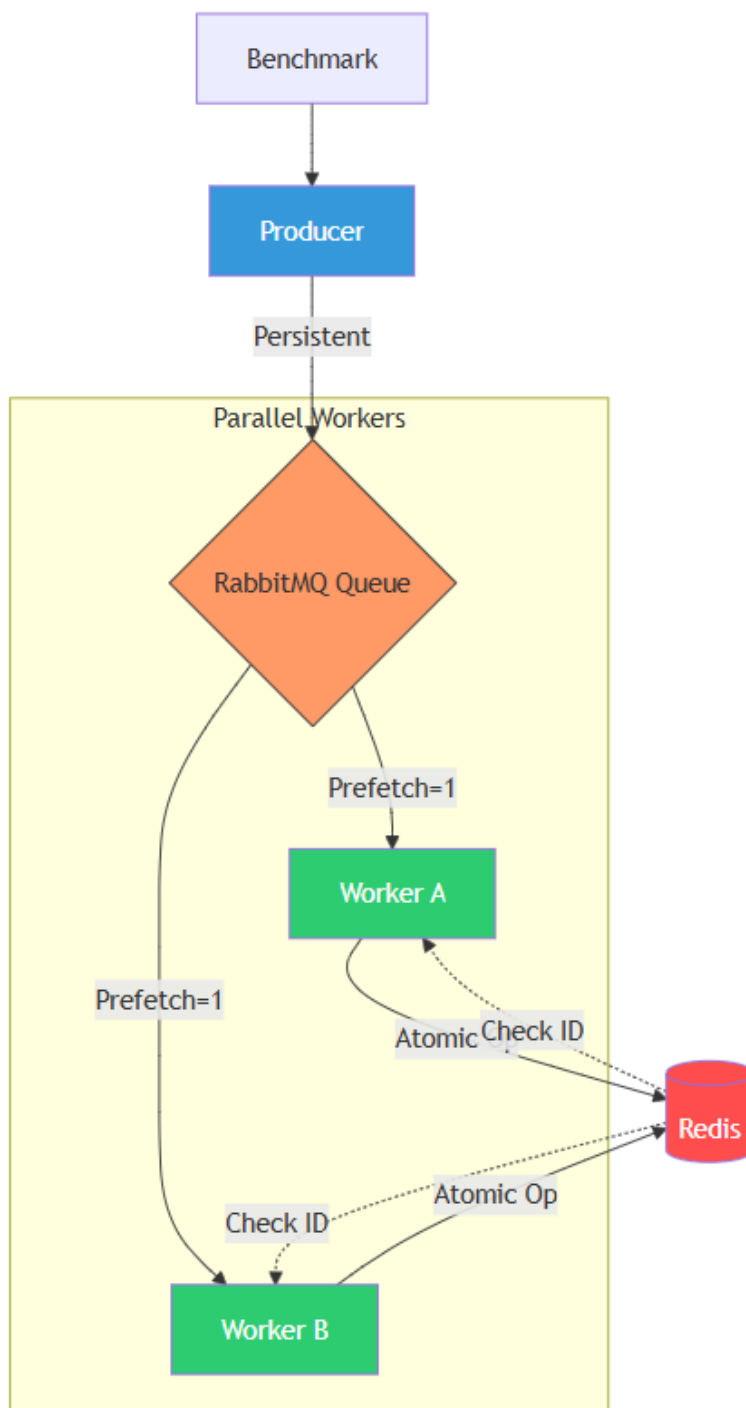


Fig. 2 - Arquitectura de Comunicación Indirecta (RabbitMQ)

En el modelo indirecto un **Producer** desacopla la carga del archivo de *benchmark* del procesamiento real y envía peticiones persistentes a una cola de RabbitMQ “cola_tickets”. El sistema escala horizontalmente mediante múltiples **Workers** que, bajo una política de Fair Dispatch (**Prefetch=1**) y confirmaciones manuales (**ACK**), consumen mensajes de

forma equilibrada, asegurando que ninguna tarea se pierda si un nodo falla. Finalmente, todos los Workers convergen en una base de datos Redis, donde se ejecutan operaciones atómicas (SET NX / INCR) y validaciones de idempotencia mediante el `request_id`, garantizando así la consistencia absoluta del inventario y evitando la sobreventa de tickets incluso en escenarios de alta contención o reintentos de red.

Fire and forget:

El patrón Fire-and-Forget se implementa en `producer.py`. El productor lee el archivo de benchmark línea a línea, construye un mensaje JSON con los datos de la compra y lo publica en la 'cola_tickets' de RabbitMQ con `delivery_mode=2` (persistente).

Una vez publicado el mensaje, el productor continúa con la siguiente línea sin esperar ninguna respuesta del procesamiento.

El desacoplamiento es total: el productor desconoce cuántos Workers hay, o el estado de la petición. RabbitMQ actúa como buffer almacenando los mensajes de forma persistente en disco hasta que los Workers los consuman.

2.3 Consistencia, Redis.

Para garantizar la integridad del inventario, el sistema delega la exclusividad de las ventas a Redis, actuando frente a los múltiples Workers.

En el caso de asientos numerados, el código utiliza la instrucción `set(clave, valor, nx=True)`, donde el parámetro NX asegura que la reserva solo sea exitosa si el asiento no ha sido ocupado previamente por otro proceso.

Para los tickets no numerados, se emplea la función `incr(contador)`, que permite gestionar el aforo de forma segura mediante incrementos atómicos que evitan condiciones de carrera.

Finalmente, la idempotencia se asegura mediante un registro de `request_id` en Redis: antes de confirmar cualquier operación, el Worker verifica si dicho identificador ya existe, evitando que reintentos de red de RabbitMQ provoquen ventas duplicadas y garantizando que el estado final de la base de datos sea siempre consistente.

3. Despliegue (2 máquinas)

El sistema se ha desplegado en dos máquinas conectadas en red local:

- **Portátil (Servidor):**
 - Name Server (Pyro).
 - Frontend y workers (comunicación directa).
 - Broker RabbitMQ y workers (comunicación indirecta).
 - Redis como backend de datos.
- **PC (Cliente):**
 - Cliente encargado de ejecutar los benchmarks.

- **Configuración**

Se ha utilizado la IP privada del portátil (por ejemplo, 192.168.1.131) para permitir la comunicación entre máquinas, evitando el uso de 0.0.0.0 o 127.0.0.1 (modificado desde el cliente y el servidor en todos los archivos

- **Comunicación directa (Pyro)**
 1. Se inicia el Name Server.
 2. Se ejecuta el frontend.
 3. Se lanzan varios workers, que se registran dinámicamente.
 4. El cliente envía peticiones al frontend, que las distribuye entre los workers.
- **Comunicación indirecta (RabbitMQ)**
 1. Se inicia el broker RabbitMQ.
 2. Se lanzan los workers consumidores.
 3. El cliente (producer) envía peticiones a la cola.

Prueba de escalado dinámico

Además de las ejecuciones con un número fijo de workers, se realizó una prueba de escalado dinámico, documentada en vídeo. En esta prueba, el sistema se inició con un único worker y, durante la ejecución del benchmark, se añadieron nuevos workers sin detener el sistema.

En la arquitectura directa, el frontend Pyro detectó los nuevos workers registrados en el Name Server y comenzó a repartir peticiones entre ellos sin modificar ni reiniciar el cliente.

En la arquitectura indirecta, RabbitMQ empezó a distribuir automáticamente los mensajes pendientes entre los nuevos workers consumidores, reduciendo el tiempo de vaciado de la cola.

Esta prueba demuestra que el sistema puede añadir capacidad de procesamiento durante la ejecución, manteniendo la corrección de las compras y sin interrumpir el servicio.

4. Resultados y gráficas

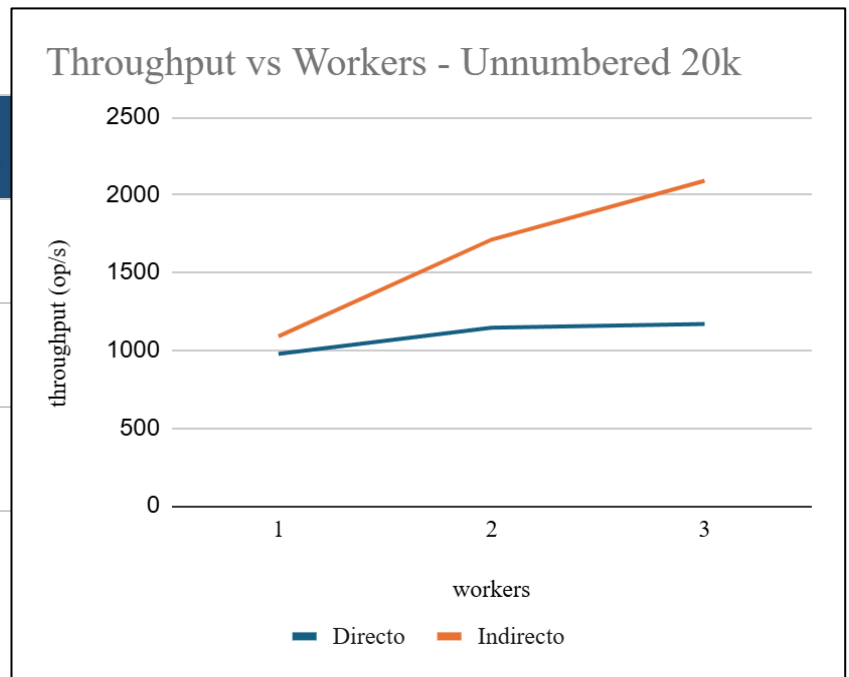
A continuación se presentan las gráficas de rendimiento obtenidas para cada benchmark en ambas arquitecturas. Todas las gráficas enfrentan el throughput (ops/seg) y el tiempo total (segundos) frente al número de Workers activos (1, 2 y 3 para indirecto; 1, 2 y 3 para directo), permitiendo observar la escalabilidad de cada modelo. Los datos numéricos completos se recogen en la siguiente tabla:

modelo	benchmark	workers	tiempo_envio	tiempo_total	throughput_envio	throughput_total	success	fail
indirecto	benchmark_unnumbered_20000.txt	1	1,72	18,35	11603,76	1089,88	20000	0
indirecto	benchmark_numbered_60000.txt	1	1,37	21,98	19006,16	1182,53	20000	5997
indirecto	bm_hotspot.txt	1	2,84	48,06	21132,21	1248,44	10008	49992
indirecto	bm_uniforme.txt	1	2,59	45,45	23180,89	1320,12	19022	40978
indirecto	benchmark_unnumbered_20000.txt	2	1,16	11,68	17190,46	1712,16	20000	0
indirecto	benchmark_numbered_60000.txt	2	1,35	14,37	19279,05	1809,22	20000	5997
indirecto	bm_hotspot.txt	2	3,73	30,95	16090,11	1938,35	10008	49992
indirecto	bm_uniforme.txt	2	2,70	30,39	22204,87	1974,53	19022	40978
indirecto	benchmark_unnumbered_20000.txt	3	1,01	9,56	19859,86	2091,13	20000	0
indirecto	benchmark_numbered_60000.txt	3	1,38	11,79	18842,59	2205,31	20000	5997
indirecto	bm_hotspot.txt	3	2,77	24,26	21680,28	2473,56	10008	49992
indirecto	bm_uniforme.txt	3	2,84	23,88	21130,81	2512,89	19022	40978
directo	benchmark_unnumbered_20000.txt	1		20,47		977,03	20000	0
directo	benchmark_numbered_60000.txt	1		21,68		1199,22	20000	5997
directo	bm_hotspot.txt	1		54,56		1099,75	10008	49992
directo	bm_uniforme.txt	1		54,17		1107,58	19022	40978
directo	benchmark_unnumbered_20000.txt	2		17,46		1145,43	20000	0
directo	benchmark_numbered_60000.txt	2		19,54		1330,17	20000	5997
directo	bm_hotspot.txt	2		45,19		1327,87	10008	49992
directo	bm_uniforme.txt	2		46,28		1296,39	19022	40978
directo	benchmark_unnumbered_20000.txt	3		17,11		1169,16	20000	0
directo	benchmark_numbered_60000.txt	3		21,15		1228,96	20000	5997
directo	bm_hotspot.txt	3		48,46		1238,15	10008	49992
directo	bm_uniforme.txt	3		48,93		1226,16	19022	40978

- **Throughput vs workers**

Unnumbered 20k

Workers	Directo	Indirecto
1	977,03	1089,88
2	1145,43	1712,16
3	1169,16	2091,13



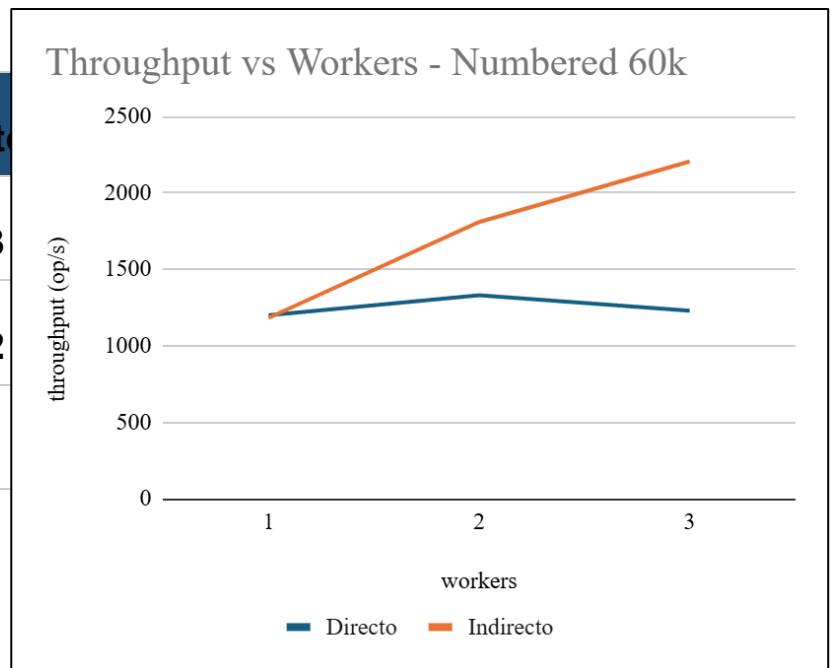
Se muestran 20.000 peticiones de compra para un aforo de 20.000 asientos sin numerar, por lo que todas son exitosas (success=20.000, fail=0).

En el modelo Directo (Pyro4) el throughput crece de forma moderada al añadir Workers: de 977 ops/seg con 1 Worker a 1.169 ops/seg con 3 Workers, debido a la naturaleza síncrona del RPC que obliga al cliente a esperar cada respuesta.

En el modelo Indirecto (RabbitMQ) el throughput es significativamente mayor: 1.090 ops/seg con 1 Worker, 1.712 ops/seg con 2 Workers y 2.091 ops/seg con 3 Workers. La asincronía del Fire-and-Forget permite que el productor inunde la cola sin bloquearse, y los Workers consuman a su ritmo máximo al no haber contención por asientos específicos.

Numbered 60k

Workers	Directo	Indirecto
1	1199,22	1182,53
2	1330,17	1809,22
3	1228,96	2205,31



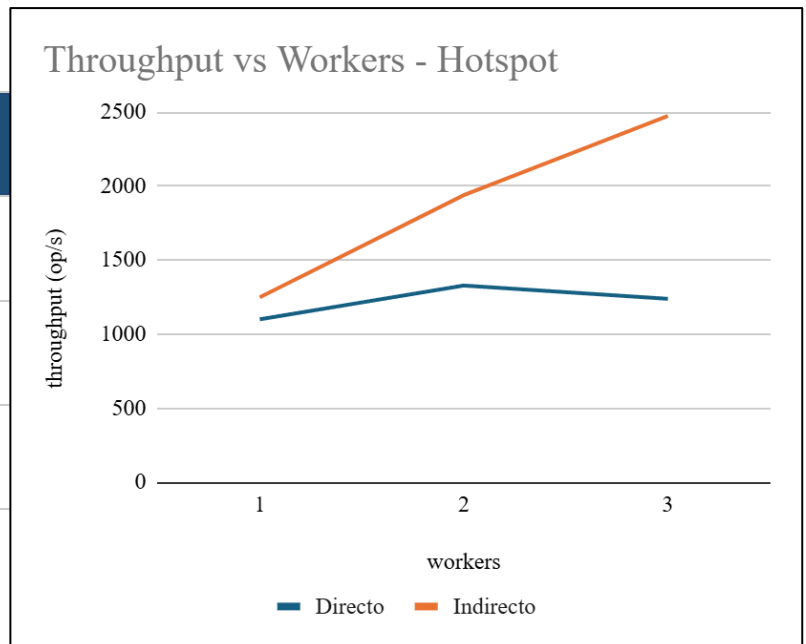
25.997 peticiones para 20.000 asientos numerados, donde todas las primeras peticiones comptien por el asiento 42 (hotspot). Se venden 20.000 asientos únicos y 5.997 son rechazados por estar ya vendidos (success=20.000, fail=5.997).

En el modelo Directo, el throughput sube de 1.199 a 1.229 ops/seg al pasar de 1 a 3 Workers, con tiempos de 21,68 s a 21,15 s. El incremento es modesto porque las peticiones rechazadas se procesan al mismo ritmo que las exitosas sin ahorro de tiempo.

En el modelo Indirecto, el throughput se dispara de 1.183 a 2.205 ops/seg y el tiempo baja de 21,98 s a 11,79 s al añadir Workers. La asincronía permite que mientras unos Workers esperan la respuesta de Redis para el asiento en disputa, otros procesen peticiones sobre asientos distintos sin bloquearse.

Hotspot

Workers	Directo	Indirecto
1	1099,75	1248,44
2	1327,87	1938,35
3	1238,15	2473,56



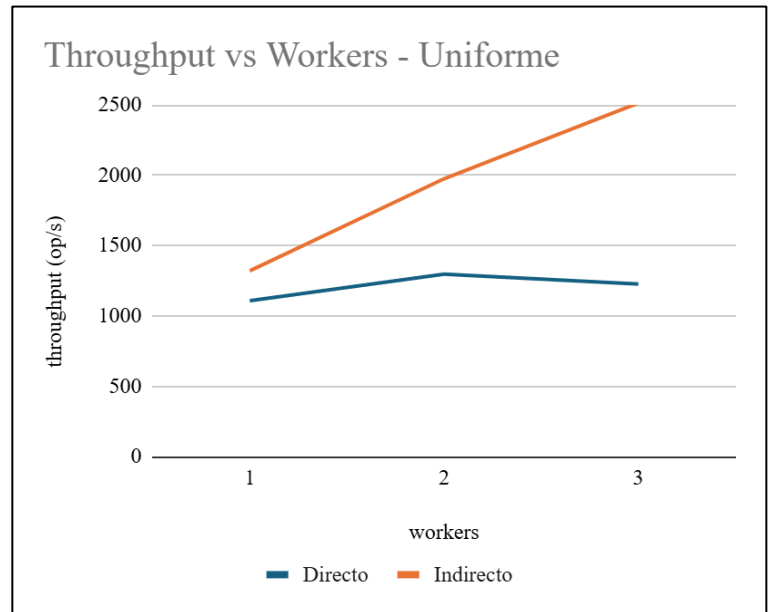
60.000 peticiones donde el 80 % se concentra en el 5 % de los asientos (asientos 1–1.000). Se consiguen ~10.008 compras exitosas y 49.992 rechazos (success=10.008, fail=49.992).

En el modelo Directo, el throughput escala de 1.100 a 1.238 ops/seg al pasar de 1 a 3 Workers (tiempo de 54,56 s a 48,46 s). Las peticiones que chocan contra los hotspots son rechazadas instantáneamente por Redis (SET NX), liberando rápido la llamada RPC.

En el modelo Indirecto, el throughput se duplica de 1.248 a 2.474 ops/seg y el tiempo se reduce de 48,06 s a 24,26 s. Disponer de más Workers ayuda a procesar y descartar más rápido el exceso de peticiones sobre los asientos calientes.

Uniforme

Workers	Directo	Indirecto
1	1107,58	1320,12
2	1296,39	1974,53
3	1226,16	2512,89



60.000 peticiones distribuidas uniformemente entre 20.000 asientos. Se esperan ~19.022 compras exitosas y ~40.978 rechazos (success=19.022, fail=40.978). En el modelo Directo, el throughput sube de 1.108 a 1.296 ops/seg al pasar de 1 a 3 Workers (de 54,17 s a 46,28 s).

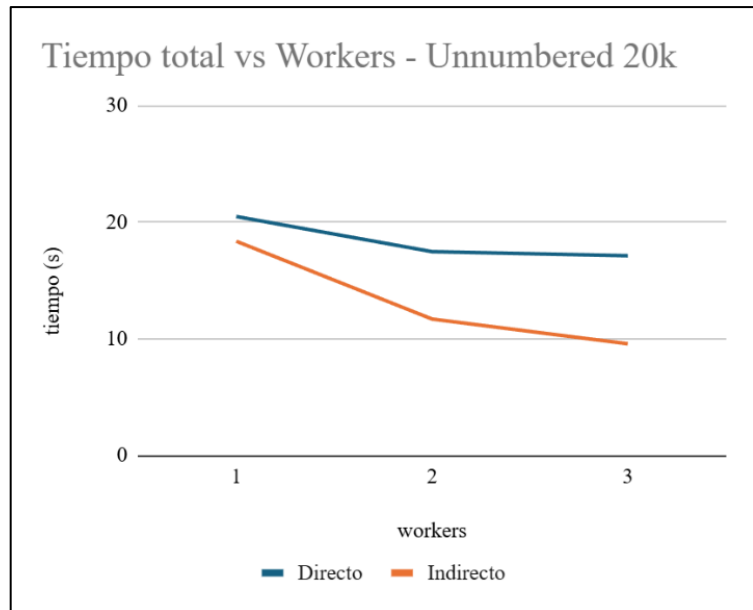
La distribución uniforme evita bloqueos centralizados y Redis resuelve las colisiones al instante, pero la naturaleza síncrona limita la mejora al añadir Workers.

En el modelo Indirecto, el throughput escala de 1.320 a 2.513 ops/seg y el tiempo baja de 45,45 s a 23,88 s.

- **Tiempo vs Workers**

Unnumbered 20k

Workers	Directo	Indirecto
1	20,47	18,35
2	17,46	11,68
3	17,11	9,56



Entradas no numeradas (20.000 peticiones)

El tiempo total del modelo Directo se reduce de 20,47 s (1 Worker) a 17,11 s (3 Workers), una mejora del 16 %.

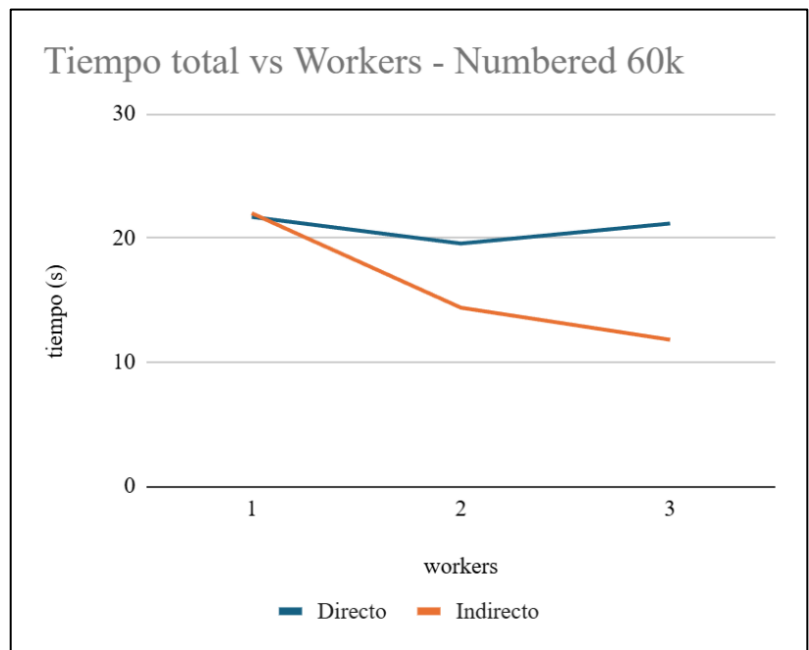
El modelo Indirecto pasa de 18,35 s a 9,56 s, reduciendo el tiempo total en torno a un 48 %. Esta mejora es más notable porque RabbitMQ permite desacoplar el envío de peticiones del procesamiento real, y los workers pueden consumir mensajes en paralelo.

La ausencia de contención en asientos no numerados permite que el modelo Indirecto aproveche al máximo el paralelismo, reduciendo drásticamente los tiempos.

Con los datos obtenidos, se observa que el modelo indirecto reduce más claramente el tiempo al aumentar de 1 a 3 workers, mientras que el modelo directo mejora de forma más limitada.

Numbered 60k

Workers	Directo	Indirecto
1	21,68	21,98
2	19,54	14,37
3	21,15	11,79



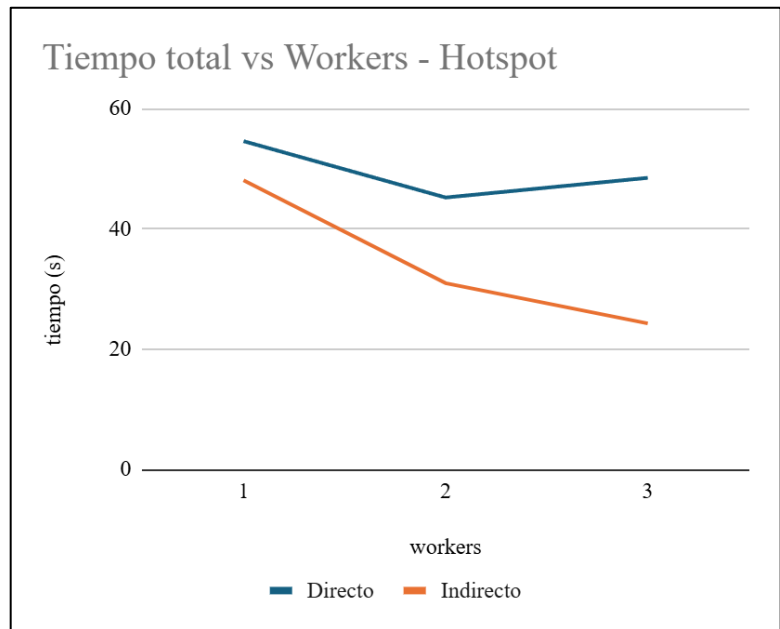
Entradas numeradas con hotspot asiento 42 (25.997 peticiones)

El modelo Directo se mantiene prácticamente estable (21,68 s a 21,15 s), ya que el hotspot en el asiento 42 provoca que todos los Workers compitan por el mismo recurso en Redis, limitando la ganancia de paralelismo.

El modelo Indirecto reduce el tiempo de 21,98 s a 11,79 s (-46 %). Con los datos obtenidos, se observa que el modelo indirecto reduce más claramente el tiempo al aumentar de 1 a 3 workers, mientras que el modelo directo mejora de forma más limitada.

Hotspot

Workers	Directo	Indirecto
1	54,56	48,06
2	45,19	30,95
3	48,46	24,26



Hotspot 80/5 (60.000 peticiones)

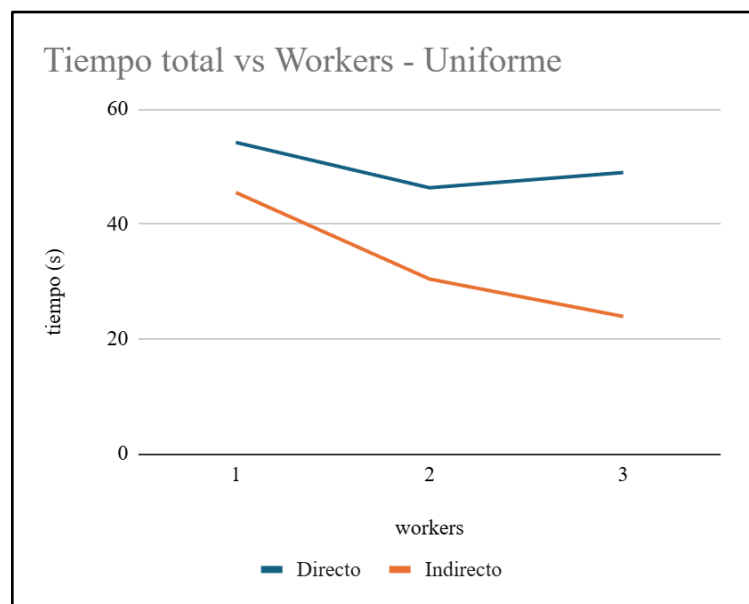
El tiempo del modelo Directo mejora ligeramente de 54,56 s a 48,46 s (-11 %).

El modelo Indirecto lo reduce de 48,06 s a 24,26 s (-50 %), los Workers descartan rápidamente las colisiones sobre los asientos calientes.

Con los datos obtenidos, se observa que el modelo indirecto reduce más claramente el tiempo al aumentar de 1 a 3 workers, mientras que el modelo directo mejora de forma más limitada.

Uniforme

Workers	Directo	Indirecto
1	54,17	45,45
2	46,28	30,39
3	48,93	23,88



Distribución uniforme (60.000 peticiones) El modelo Directo reduce el tiempo de 54,17 s a 46,28 s (-15 %). El modelo Indirecto lo reduce de 45,45 s a 23,88 s (-47 %).

5. Conclusiones

La comunicación directa con Pyro4 ofrece un modelo sencillo y predecible que escala adecuadamente para cargas moderadas. Sin embargo, su naturaleza síncrona la hace vulnerable a fallos y poco eficiente para picos de demanda.

En nuestro despliegue, Redis, RabbitMQ, el frontend y los workers se ejecutaron en el portátil servidor, mientras que el PC actuó como cliente encargado de lanzar los benchmarks (peticiones).

La comunicación indirecta con RabbitMQ demuestra ser superior en rendimiento y resiliencia, especialmente para entradas no numeradas donde no hay contención por recursos. Para entradas numeradas con hotspots, el cuello de botella es Redis, no el middleware, por lo que añadir más workers no mejora el throughput linealmente. En el despliegue en 2 máquinas, RabbitMQ y Redis residen en el servidor y el Producer (cliente) envía las peticiones de forma asíncrona sin bloquearse.

En conclusión, para la venta de entradas de alta demanda en entornos reales con despliegue en 2 máquinas, se recomienda la arquitectura indirecta (RabbitMQ) debido a su mayor rendimiento, mejor resiliencia (persistencia de mensajes, ACKs) y capacidad para manejar picos de tráfico. La arquitectura directa (Pyro4) es adecuada para sistemas más simples con carga moderada donde se prioriza la facilidad de implementación sobre el rendimiento bruto.